Run the flow by pressing Shift-F5 within the VS Code prompt flow visual editor. You should see output similar to that shown in listing 10.5.

Now open the `zero_shot.jinja2` prompt as shown in listing 10.6. The prompt is simple and uses no examples to extract the sentiment from the text. What is especially interesting to note is that the prompt doesn't even mention the phrase sentiment, and the LLM seems to understand the intent.

**Listing 10.6 `zero_shot.jinja2`**

```
system:
Classify the text into neutral, negative or positive.
Return on the result and nothing else.     #1

user:
{{statement}}     #2
```

**#1 Provides essential guidance on performing the classification**
**#2 The statement of text to classify**

Zero-shot prompt engineering is about using the ability of the LLM to generalize broadly based on its training material. This exercise demonstrates how knowledge within the LLM can be put to work for other tasks. The LLM's ability to self-contextualize and apply knowledge can extend beyond its training. In the next section, we extend this concept further by looking at how LLMs can reason.

## 10.2 Reasoning in prompt engineering

LLMs like ChatGPT were developed to function as chat completion models, where text content is fed into the model, whose responses align with completing that request. LLMs were never trained to reason, plan, think, or have thoughts.

However, much like we demonstrated with the examples in the previous section, LLMs can be prompted to extract their generalities and be extended beyond their initial design. While an LLM isn't designed to reason, the training material fed into the model provides an understanding of reasoning, planning, and thought. Therefore, by extension, an LLM understands what reasoning is and can employ the concept of reasoning.